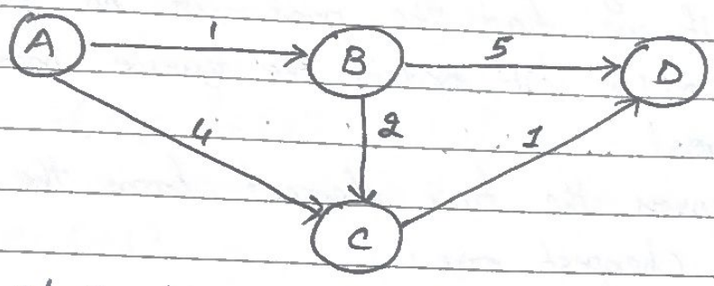
# Lab-3 Uniform Cost Search

To implement the Uniform Cost Search algorithm to find the least cost path between a start node and a goal node in a weighted graph.




The edges have costs. Find the cheapest path from A to D.

Step 1 : Declare a weighted graph

```
graph = {'A' : [('B', 1), ('C', 4)],
         'B' : [('D', 5), ('C', 2)],
         'C' : [('D', 1)],
         'D' : []}
```

A dictionary named graph is created. It stores all the nodes and their neighbouring nodes.
A weighted graph stores pairs : {'A' : [('B', 1), ('C', 4)]}
From A we can goto B with cost 1 or C with cost 4.

Step 2: Function Definition

```
def ucs (start, goal) :
```

The function accepts the starting node and goal node

Step 3: Initialize Queue

```
queue = [(0, [start])]
```

The starting cost is 0 because we have not travelled anywhere.
We are currently at A and have spent 0 cost.

classmate
Date
Page

## Step 4: Sort Queue and take cheapest

```
while queue:
    queue.sort()
    cost, path = queue.pop(0)
```

Continue searching while the queue is not empty. The loop stops if we find the goal or no path exists. By using sort(), it orders the queue from cheapest to the dearest.
pop(0) removes the first element from the queue which will be the cheapest one.

## Step 5: Goal Test

```
if path[-1] == goal:
    return cost, path
```

Goal test on pop and never on generate. UCS does not stop at the first goal it finds, it stops at the first goal it pops.

## Step 6: Expand Current Node

```
for n, w in graph[path[-1]]:
    queue.append((cost+w, path+[n]))
```

The loop unpacks the g node neighbour and the edge weight of the current node.
Adds a new path to the queue

## Step 7: Return Failure

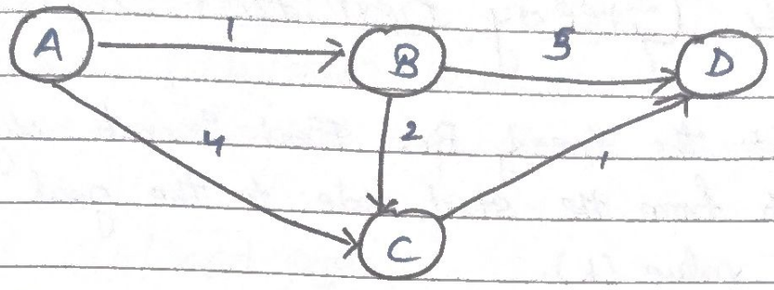
```
return None
```

When there does not exists any path from source to goal.

## Step 8: Call and Print

```
print("UCS:", ucs('A', 'D'))
```

For the Graph




Initially: Queue [empty] Visited [empty]

[(0, [A])]

Remove A and Expand A (i.e. cheapest)

(0, [A])

Add

(1, (A, B))

(4, (A, C))

Remove (1, (A, B)) i.e cheapest and Expand

Add

(3, (A, B, C))

(6, (A, B, D))

Remove (3, (A, B, C)) i.e cheapest & expand

Add

(4, (A, B, C, D))

(6, (A, B, D))

Remove (4, (A, B, C, D)) i.e cheapest

Goal node reached

=> cost = 4

=> path = ['A', 'B', 'C', 'D']